

Linguaggi Formali e Compilatori

Argomento 06:

La Java Virtual Machine

Prof. Giuseppe Psaila

Laurea Magistrale in Ingegneria Informatica
Università di Bergamo

- Java nasce per essere un linguaggio di programmazione **indipendente** dalla piattaforma di esecuzione (hardware/software)
- Inizialmente era stato pensato per i **micro-controllori**
- ma ha avuto molto più successo in ambiti con maggiore potenza di calcolo

Obiettivo

- Compilare il programma una volta
- Distribuirlo ed eseguirlo su piattaforme diverse
- Java è un linguaggio pensato come
Write Once - Run Anywhere

Approccio

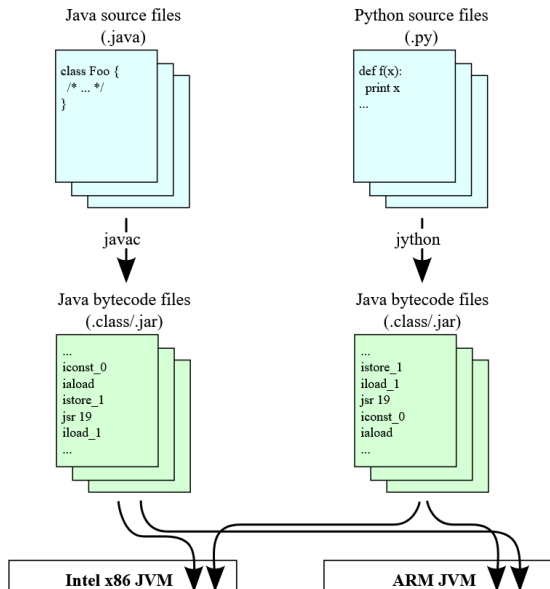
- Tradurre il programma in una sorta di “linguaggio macchina” **indipendente** (chiamato **ByteCode**)
- Eseguire il programma tradotto con un **motore di esecuzione** specifico per la piattaforma, una specie di **CPU virtuale** (chiamata **Java Virtual Machine**)

Perché ByteCode?

- Perché il codice operativo (cioé l'istruzione macchina) richiede 1 Byte
- 256 codici operativi sembrano tanti, ma in realtà non lo sono.
- La JVM è come un processore RISC

Solo Java?

- La JVM nasce per Java, ma è indipendente
- Altri linguaggi?
 - Jython (versione compilata di Python)
 - Kotlin
 - Scala
 - Groovy



La Java Virtual Machine

- La JVM ufficiale è di Oracle (in origine di Sun Microsystems)
- Esistono anche JVM non ufficiali (in passato, mi sono imbattuto nella JVM della Fondazione Gnu, molto inefficiente)

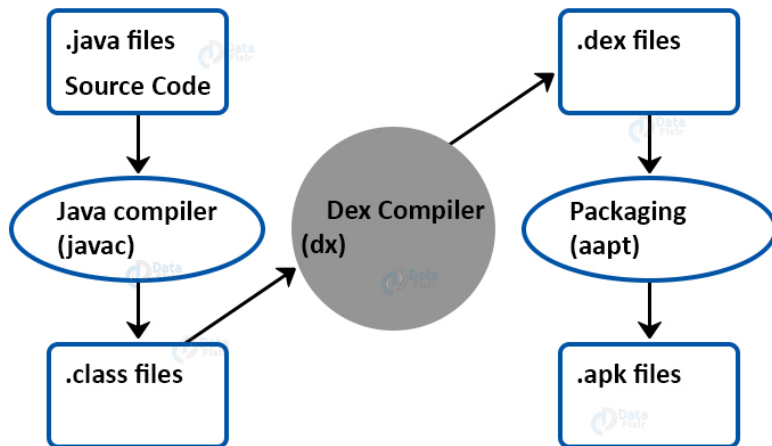
Android

- Le app per Android sono scritte in Java
- Ma Android non usa la JVM ufficiale, usa una **Process Virtual Machine** chiamata **Dalvik**
- Dalvik è un piccolo porto di pescatori in Islanda, dove vissero alcuni antenati di Dan Bomstein, creatore della Dalvik

Dalvik

- Dalvik non usa il ByteCode
- usa una versione modificata, con codici operati a 16 bit
- Lo scopo è ridurre le operazioni di calcolo, perché con 16 bit il numero di istruzioni disponibili aumenta (passa da RISC a CISC) (meno istruzioni, meno memoria occupata dal codice)
- Il compilatore è sempre javac, quindi genera ByteCode
- Il codice viene poi trasformato nel formato DEX (codice operativo a 16 bit)

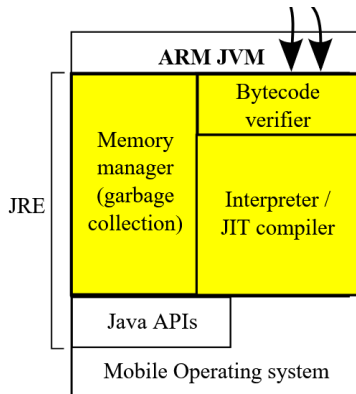
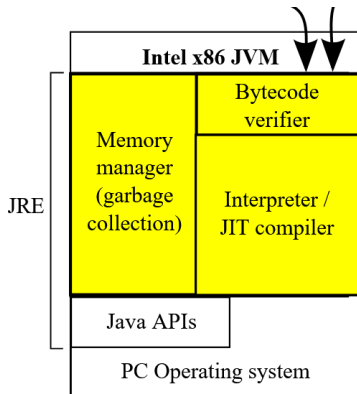
Working of Dex Compiler



Ambiente di Esecuzione

- **JRE** - Java Runtime Environment
- Include la JVM specifica per la piattaforma hardware/software

Architettura



Byte-Code Verifier

- **Inherently Safe:** nessun programma utente può mandare in crash la macchina ospite
- Il ByteCode Verifier verifica il codice prima della sua esecuzione
- Che cosa verifica?

Byte-Code Verifier - Verifiche

- I salti devono avvenire su posizioni valide
- I dati sono sempre inizializzati e i riferimenti sono sempre corretti dal punto di vista dei tipi
- L'accesso a dati **private** o **package private** è **rigorosamente** controllato

JIT

- **JIT**: Just-In-Time COmpiler
traduce il byte Code nel linguaggio macchina della
piattaforma hardware

Modello di Funzionamento

Java Stack

- Un **Java Stack** è associato ad **ogni thread** attivo nella JVM

In questo stack vengono impilati i **Frame**, ossia i **record di attivazione**

- Un **Frame** è così composto:

- **Local-Variable Array**

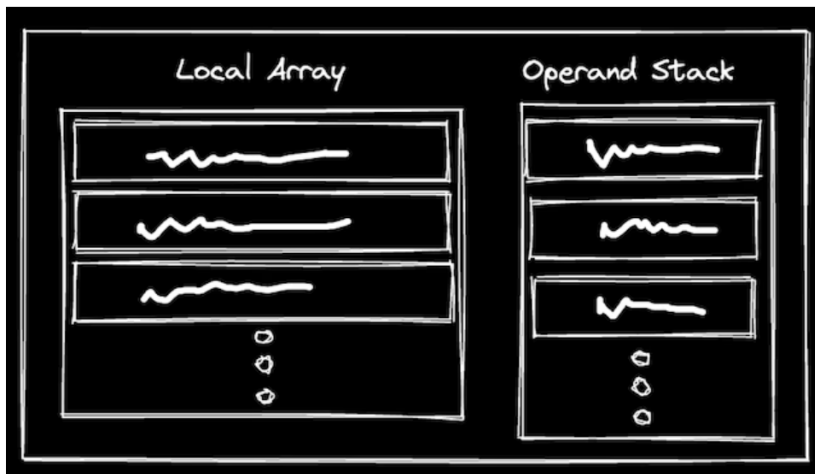
Operand Stack

Program Counter

Pointer alla classe nella quale il metodo è definito

Pointer al frame del metodo invocante

Method frame



Frame - Local-Variable Array

- Memorizza le variabili locali del metodo;
Le variabili vengono gestite tramite la loro posizione nell'array
- Ogni variabile è rappresentata da un valore a 32 bit (reference per gli oggetti, il valore per i tipi built-in)
- Anche i parametri formali sono in questo array e il passaggio avviene **per valore/copia**

Frame - Operand Stack

- Ogni elemento dello stack occupa 32 bit
- Serve per memorizzare i risultati intermedi delle chiamate di metodo
- In generale, vengono impilati gli operandi delle espressioni e i calcoli vengono svolti qui
- Il ByteCode fornisce istruzioni per trasferire valori da/verso l'Operand Stack

Frame - Dimensioni di Local-Variable Array e Operand Stack

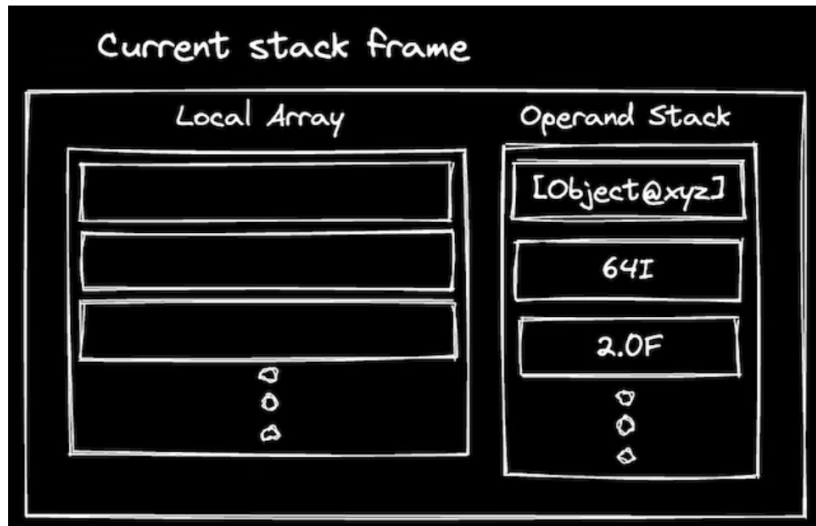
- Il compilatore Javac calcola la dimension per entrambi
- Quindi, i frame sono generati con questa dimensione cablata nel ByteCode

Chiamata di un Metodo (1/3)

Sull'Operand Stack del metodo Chiamante

- Si fa il **push** dell'ObjectRef dell'oggetto sul quale si invoca il metodo
(se il metodo è `static`, questo ObjectRef non viene impilato)
- Si fa il **push** degli n parametri attuali del metodo invocato

Invoking-Method frame



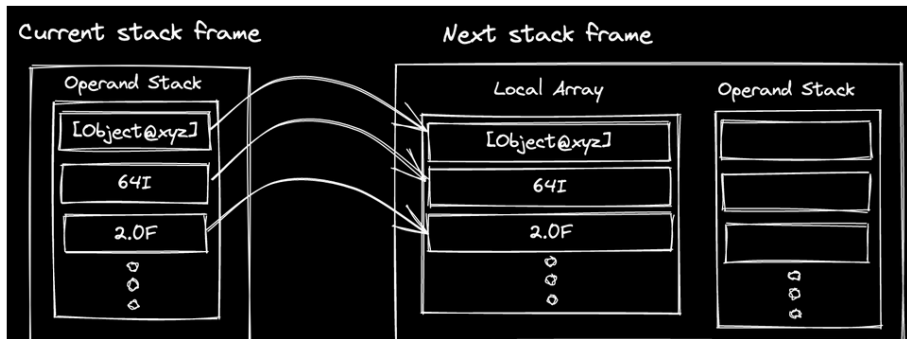
Chiamata di un Metodo (2/3)

Invocazione

- Creazione di un nuovo frame
- **Pop** di ObjectRef e parametri attuali dall'Operand Stack del chiamante
- Copia di ObjectRef e parametri attuali nel Variable Stack nel frame del chiamato

Esecuzione

Invocation



Chiamata di un Metodo (3/3)

Restituzione del Valore

- Il valore dell'espressione nell'istruzione `return` viene messo sull'Operand Stack (del metodo chiamato)
- **Pop** da Operand Stack del chiamato
- **Push** sull'Operand Stack del chiamante
- Il frame del metodo chiamante diventa **CURRENT**
- Il **Program Counter** del chiamante viene incrementato

Variabili e Oggetti

- **Tipi Built-in**

Sono i tipi semplici, come `int`, `float`, `boolean`

- **Oggetti**

Istanze di classi

Tipi Built-in

- byte - intero in complemento a due su 1 Byte
- short - intero in complemento a due su 16 bit
- int - intero in complemento a due su 32 bit
- long - intero in complemento a due su 64 bit
(due elementi del Local-Variable Array)
- float - numeri in virgola mobile su 32 bit,
formato **IEEE 754**
- double - numeri in virgola mobile su 64 bit,
formato **IEEE 754**
(due elementi del Local-Variable Array)

Gestione dei Tipi Built-in

- I valori built-in sono memorizzati come sono nel Local-Variable Array e nell'Operand Stack
- Quindi, quando vengono passati come parametri, viene compiuto il valore stesso

Gestione degli Oggetti

- Il Byte-Code fornisce istruzioni specifiche per gestire gli oggetti
- Gli oggetti si trovano nello **Heap** e non sono visibili dal codice
- Infatti, la JVM fornisce un **ObjectRef**, cioè un riferimento all'oggetto (non è un puntatore)
- Le istruzioni del ByteCode dedicate accedono al contenuto dell'oggetto
⇒ lo stato dell'oggetto è gestito dalla JVM in modo trasparente

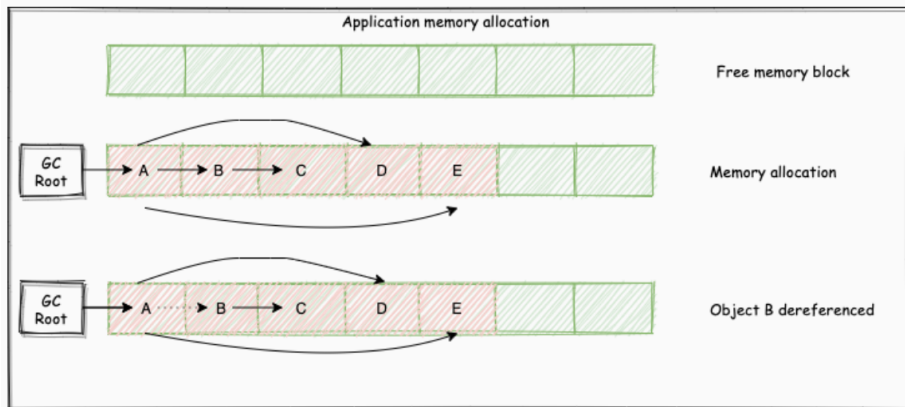
Gestione degli Oggetti

- L'**ObjectRef** è un puntatore ad una psizione all'interno di una tabella;
- il valore nella posizione indicata è l'effettiva posizione in memoria centrale dell'oggetto
- Vantaggio: il Garbage Collector può **Rilocare** gli oggetti in modo trasparente per il programma

Il Garbage Collector

- La JVM non fornisce un'istruzione per **de-allocare** gli oggetti
- Deve scoprire quali oggetti non sono più usati per rimuoverli
ogni oggetto che non viene più referenziato da variabili del programma/campi degli oggetti è spazzatura
- La tecnica usata si chiama
Mark - Sweep - Compact

Variabili e Oggetti



Il Garbage Collector

Fase 1/3

- **Mark and Sweep** - Partendo dai **riferimenti raduce** (**root references**)
- passeggia nel **grafo degli oggetti** (**object graph**) seguendo i riferimenti agli oggetti nei campi degli oggetti e nelle variabili dei metodi con frme negli stack
- Tutti gli oggetti raggiunti vengono marcati come **raggiunti**

Il Garbage Collector

Fase 2/3

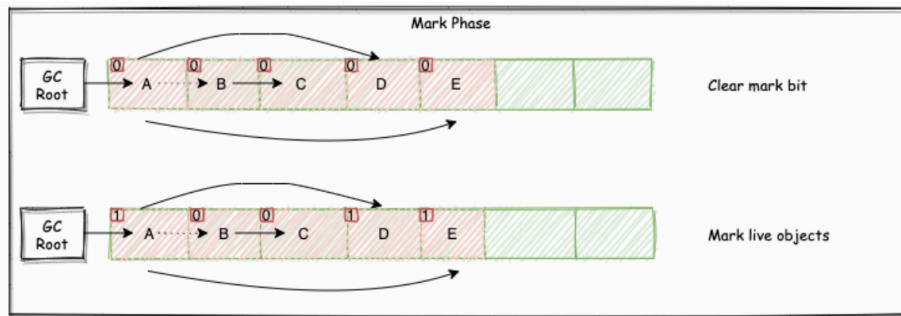
- **Remove** - Elimina dallo heap tutti gli oggetti non marcati
- Di conseguenza, lo heap si **frammenta**

Il Garbage Collector

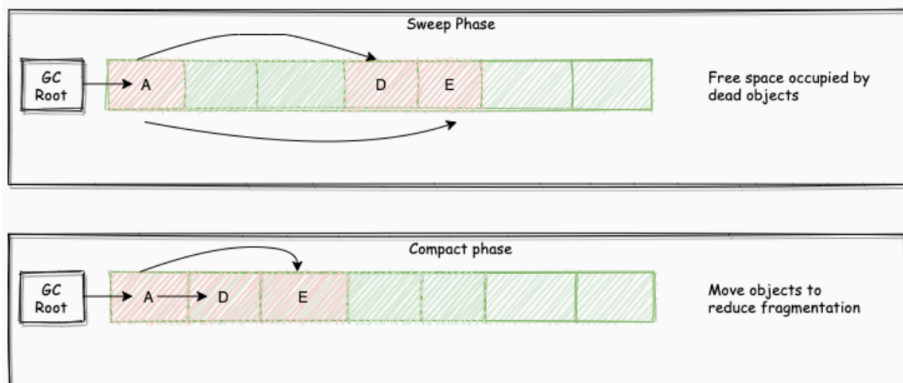
Fase 3/3

- **Compact** - Gli oggetti rimasti vengono spostati (rilocati) per togliere la frammentazione dello heap
- Se lo heap è frammentato, lo spazio diventa meno utilizzabile
- La rilocazione modifica la tabella degli ObjectRef, lasciando i riferimenti (valori delle variabili) inalterati
- Ma rilocare vuol dire copiare, quindi la rilocazione costa tempo di esecuzione

Mark and Sweep



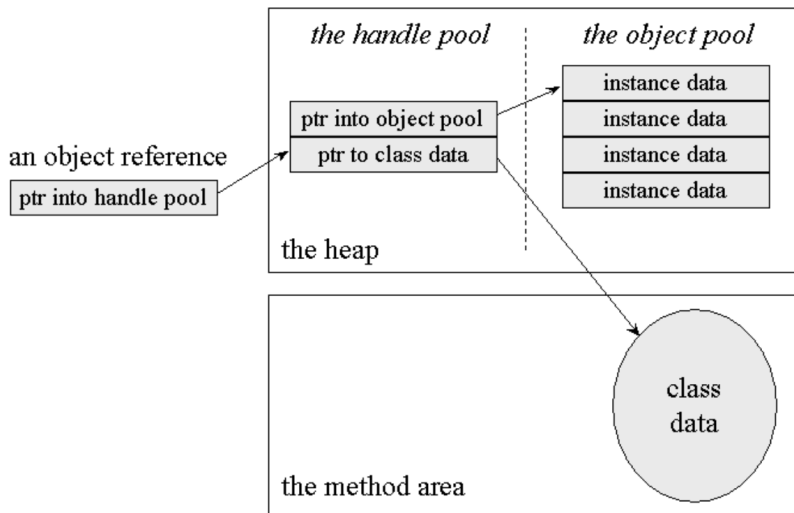
Compact



ObjectRef

- In linea generale, è un **puntatore ad un handler**
- un **handler** essenzialmente è una coppia di puntatori (pt, po)
- pt punta ad una tabella che descrive i metodi dell'oggetto, quindi punta all'oggetto di tipo **class** che descrive la classe istanziata dall'oggetto
- po è l'indirizzo nello heap dello stato dell'oggetto

Invocation



La Sincronizzazione

- Il **Multi-Threading** è una caratteristica fondamentale della JVM
- Si possono creare molti thread che vengono eseguiti in parallelo
- Esecuzione parallela = accesso concorrente a oggetti condivisi

- La JVM usa il concetto di **Monitor** per gestire l'accesso concorrente agli oggetti
- Il ByteCode fornisce due istruzioni per entrare/uscire da un monitor:
MonitorEnter
MonitorExit
- Il compilatore deve garantire che ad ogni ingresso in un monitor corrisponda sempre un'uscita

MonitorEnter *ObjectRef*

- *ObjectRef* è l'oggetto sul quale si vuole invocare un metodo `synchronized`
Viene messo nell'Operand Stack
- Ogni oggetto ha un suo monitor
- Se il monitor dell'oggetto è già occupato da un altro thread, il thread viene sospeso e accodato
- Se il monitor è libero, si entra nel monitor
- Ogni ingresso incrementa un **contatore dei lock**

MonitorExit

- Il contatore dei lock viene decrementato
- Quando si azzerà, il thread ha definitivamente liberato il monitor

Semplice, ma ...

il compilatore javac **non usa questo meccanismo!!!**

Meccanismo Utilizzato

- **Working Memory del Thread**

Ogni thread ha uno spazio di memoria nel quale copia i valori delle variabili prima di metterli nel Local-Variable- Stack

Master Memory → Thread Working Memory → Stack

- Quando il valore di una variabile cambia, il processo avviene al contrario

Stack → Thread Working Memory → Master Memory

- Variabili e oggetti sono dotati di **lock** gestiti dal **Main-Memory Manager**

Main-Memory Manager

- Serializza gli accessi alle variabili
- Serializza gli accessi ai lock

nella Master Copy nella Main Memory

Con metodi **NON** synchronized

- La Thread Working Memory è una specie di cache memory
- non essendo in competizione con altri thread, non c'è necessità di aggiornare la Master Cpy nella Master Memory
Ogni tanto lo fa, ma non sempre
- Risultato: un altro thread accede alla Master Copy che è diventata obsoleta

Con metodi `synchronized`

- Quando un thread preleva una variabile dalla Master Memory, si attiva un lock sulla master copy
- Nella sezione `synchronized`, i valori vengono prelevati e modificati nella Thread Working Memory
- Quando si esce dal metodo, i valori delle variabili si aggiornano nelle Master Copy
- Il Main-Memory Manager toglie i lock alle variabili

Qual è il vantaggio?

Vantaggio

- La gestione del lock viene fatta sulla Master Copy
Se un lock è attivo e un altro thread cerca di accedere alla Master Copy, esso viene sospeso
- Il thread che ha il lock, invece, può liberamente lavorare nella sua Thread Working Memory, in modo **indipendente** dal lock
- Quando la Master Copy viene aggiornata, si esce dalla sezione critica e il lock viene rilasciato

Variabili e Oggetti

